

第二周 递归 优秀作业

第一题

The diagram shows a recursion tree for the recurrence relation $T(n) = T(n-a) + T(a) + cn$. The root node is cn . It branches into $c(n-a)$ and $T(a)$. The $c(n-a)$ node further branches into $c(n-2a)$ and $T(a)$. This pattern continues down to a base case of $2T(a)$. A bracket on the right indicates the height of the tree is $h = (n/a) - 1$. Below the tree, the recurrence is solved using the summation formula:

$$T(n) = \sum_{i=0}^{n/a-2} [c(n-ai) + T(a)] + 2T(a)$$
$$= \frac{c}{2a} n^2 + \frac{c}{2} n - ca + \left(\frac{n}{a} + 1\right) T(a) = \Theta(n^2)$$

第二题

```
Merge_Sort(A, p, r) {  
  if p < r then { q1 <- (p + r) / 4 ; q2 <- (p + r) / 4 * 2 ; q3 <- (p + r) / 4 * 3 }  
  Merge_Sort(A, p, q1);  
  Merge_Sort(A, q1, q2);  
  Merge_Sort(A, q2, q3);  
  Merge_Sort(A, q3, r);  
  Merge(A, p, q1, q2, q3, r)  
}
```

```
Merge(A, p, q1, q2, q3, r){  
  create arrays A1 [ 1 ... , q1 - p + 1 ] and A2 [ 1 ... , q2 - q1 ] and A3 [ 1 ... , q3 - q2 ] and A4 [ 1 ... , r - q3 ]  
  for i <- 1 to q1 - p + 1 do {  
    A1 [ i ] = A [ p - 1 + i ] ;  
    A2 [ i ] = A [ q1 + i ] ;  
    A3 [ i ] = A [ q2 + i ] ;  
    A4 [ i ] = A [ q3 + i ] ;  
  }  
  i1 = i2 = i3 = i4 = 1  
  for k <- p to r do {  
    A [ k ] <- min( A1[i1], A2[i2], A3[i3], A4[i4] )  
    if A1[i1] == A[k] then i1 ++  
    if A2[i2] == A[k] then i2 ++  
    if A3[i3] == A[k] then i3 ++  
    if A4[i4] == A[k] then i4 ++  
  }  
}
```

$$O(n) = n * \log_4 n$$

时间复杂度分析:

因为分为四组递归可以视为一棵四叉树, 设原始的为层数 $I=0$, 第 i 层的时间复杂度: $T(n)=2^i * T(n/2^i)+in$, 最后该树的层数为 $(\log_4 n)+1$, 可得 $T(n)=n * T(1)+(\log_4 n)*n$, $T(1)=0$, 最后求解得 $T(n)=n * (\log_4 n)$

第三题

设解决汉诺塔问题的函数为 $Hanoi(n, A, B, C)$

当 $n=1$ 和 $n=2$ 时直接验证即可。设当 $k=n-1$ 时, 递归算法与非递归算法产生完全相同的移动序列, 则 $k=n$ 时:

将移动分为顺时针移动(S), 逆时针移动(N)和非最小圆盘塔间的移动(F)三种情况。

(1) 当 n 为奇数时, 顺时针非递归产生的移动序列为 S, F, S, F, ..., S,

逆时针非递归算法产生的序列为 N, F, N, F, ..., N;

当 n 为偶数时, 顺时针非递归产生的移动序列为 N, F, N, F, ..., N,

逆时针非递归产生的序列为 S, F, S, F, ..., S.

(2) 当 n 为奇数时, 顺时针递归算法 $Hanoi(n, A, B, C)$ 产生的移动序列为 $Hanoi(n-1, A, C, B), F, Hanoi(n-1, C, B, A)$

其中, $Hanoi(n-1, A, C, B), Hanoi(n-1, C, B, A)$ 均为偶数圆盘逆时针移动问题。由数学归纳法知, 它们产生的移动序列均为 S, F, S, F, ..., S.

因此 $Hanoi(n, A, B, C)$ 产生的移动序列为 S, F, S, F, ..., S.

当 n 为偶数时, 顺时针递归算法 $Hanoi(n, A, B, C)$ 产生的移动序列为 $Hanoi(n-1, A, C, B), F, Hanoi(n-1, C, B, A)$

其中, $Hanoi(n-1, A, C, B), Hanoi(n-1, C, B, A)$ 均为奇数圆盘逆时针移动问题。由数学归纳法知, 它们产生的移动序列均为 N, F, N, F, ..., N.

同理, 当 n 为奇数或偶数时的逆时针算法也类似。

综上所述, 由数学归纳法知, 递归算法与非递归算法产生相同的移动序列。

第四题

```
int Search (int A[], int i)
{
    if (i > n) return 0;
    if (A[i] == x) return i;
    Search (A, i+1);
}

k = Search (A, 1) // k为元素x的下标, 若A中无x, 则k=0
```